Image Features:

Scale Invariant Interest Point Detection

Scale Invariant Interest Points

How can we **independently** select interest points in each image, such that the detections are repeatable across different scales?

[Source: K. Grauman, slide credit: R. Urtasun]

Scale Invariant Interest Points

How can we **independently** select interest points in each image, such that the detections are repeatable across different scales?

[Source: K. Grauman, slide credit: R. Urtasun]

Scale Invariant Interest Points

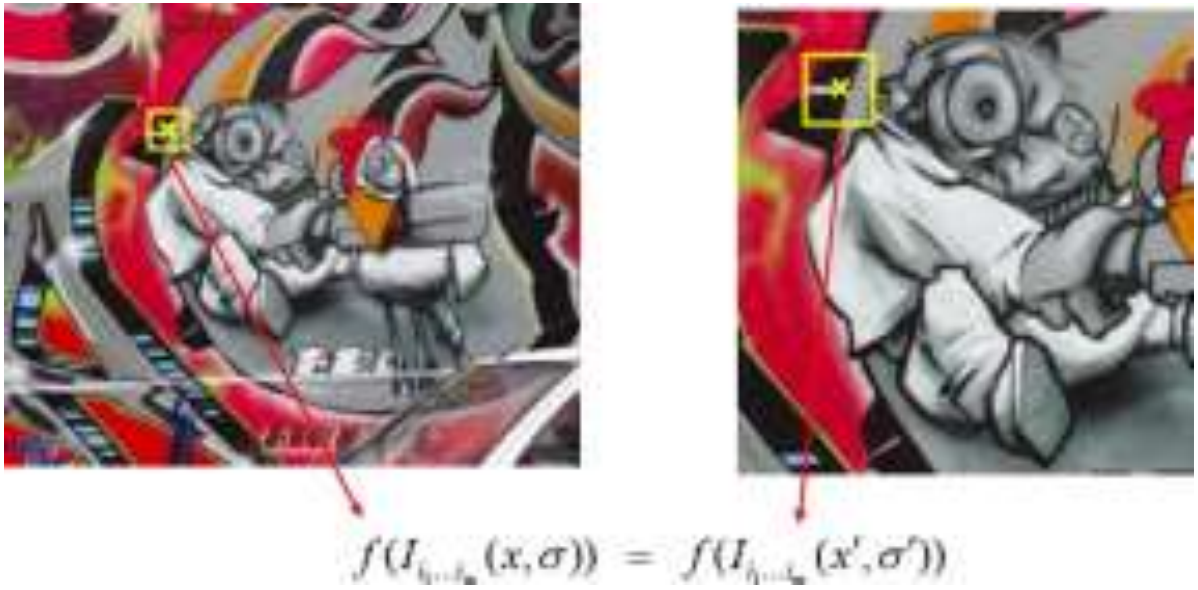
How can we **independently** select interest points in each image, such that the detections are repeatable across different scales?

- Extract features at a variety of scales, e.g., by using multiple resolutions in a pyramid, and then matching features at the same level.
- When does this work?

Scale Invariant Interest Points

How can we **independently** select interest points in each image, such that the detections are repeatable across different scales?

- More efficient to extract features that are stable in both location and scale.

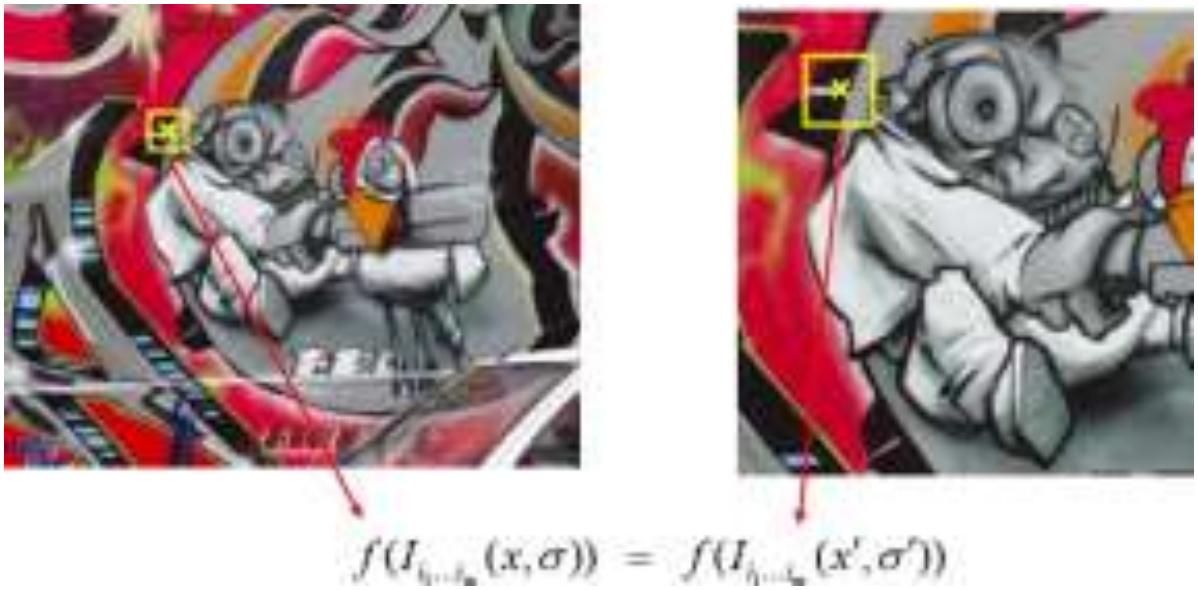


[Source: K. Grauman, slide credit: R. Urtasun]

Scale Invariant Interest Points

How can we **independently** select interest points in each image, such that the detections are repeatable across different scales?

- Find scale that gives local maxima of a function *f* in both position and scale.



[Source: K. Grauman, slide credit: R. Urtasun]

Automatic Scale Selection

Function responses for increasing scale (scale signature).

Automatic Scale Selection

Function responses for increasing scale (scale signature).

Automatic Scale Selection

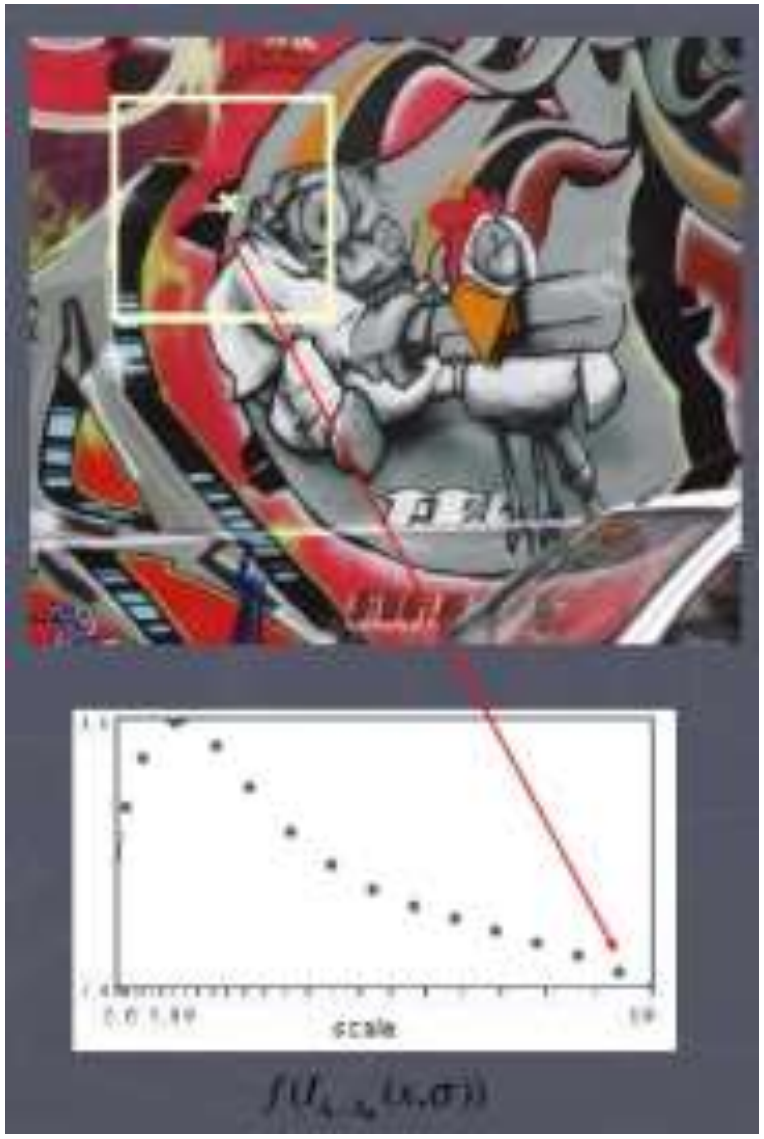
Function responses for increasing scale (scale signature).

Automatic Scale Selection

Function responses for increasing scale (scale signature).

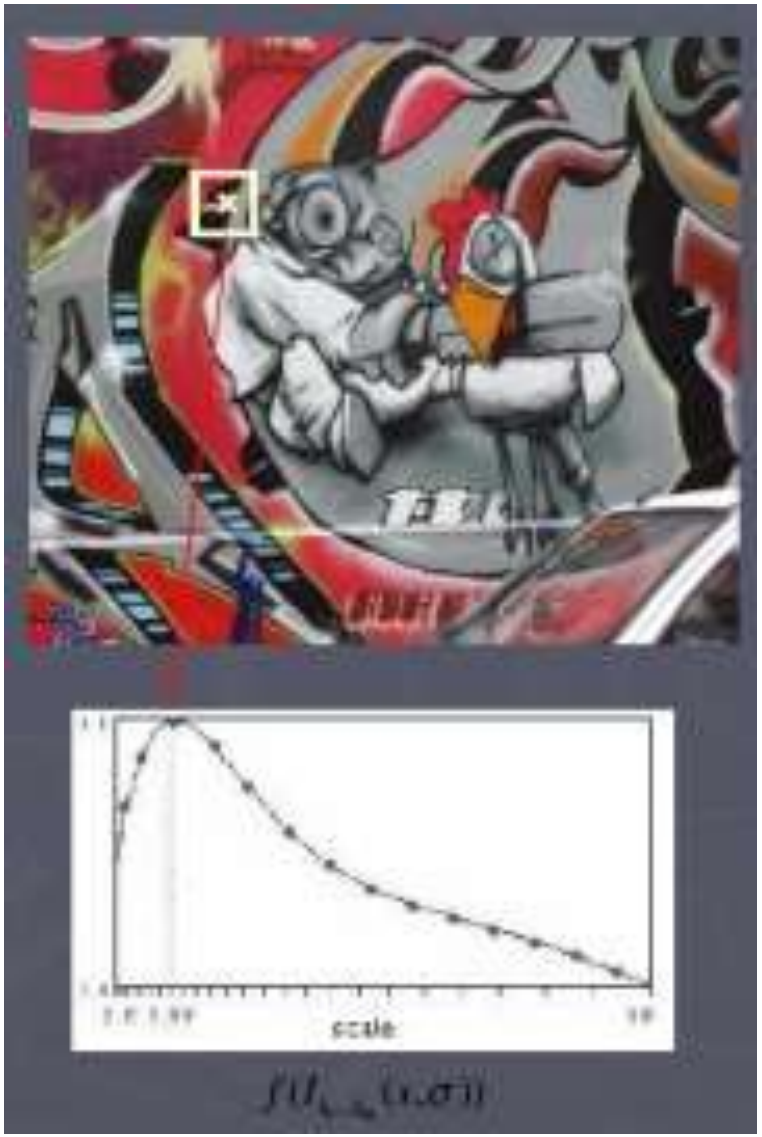
Automatic Scale Selection

Function responses for increasing scale (scale signature).



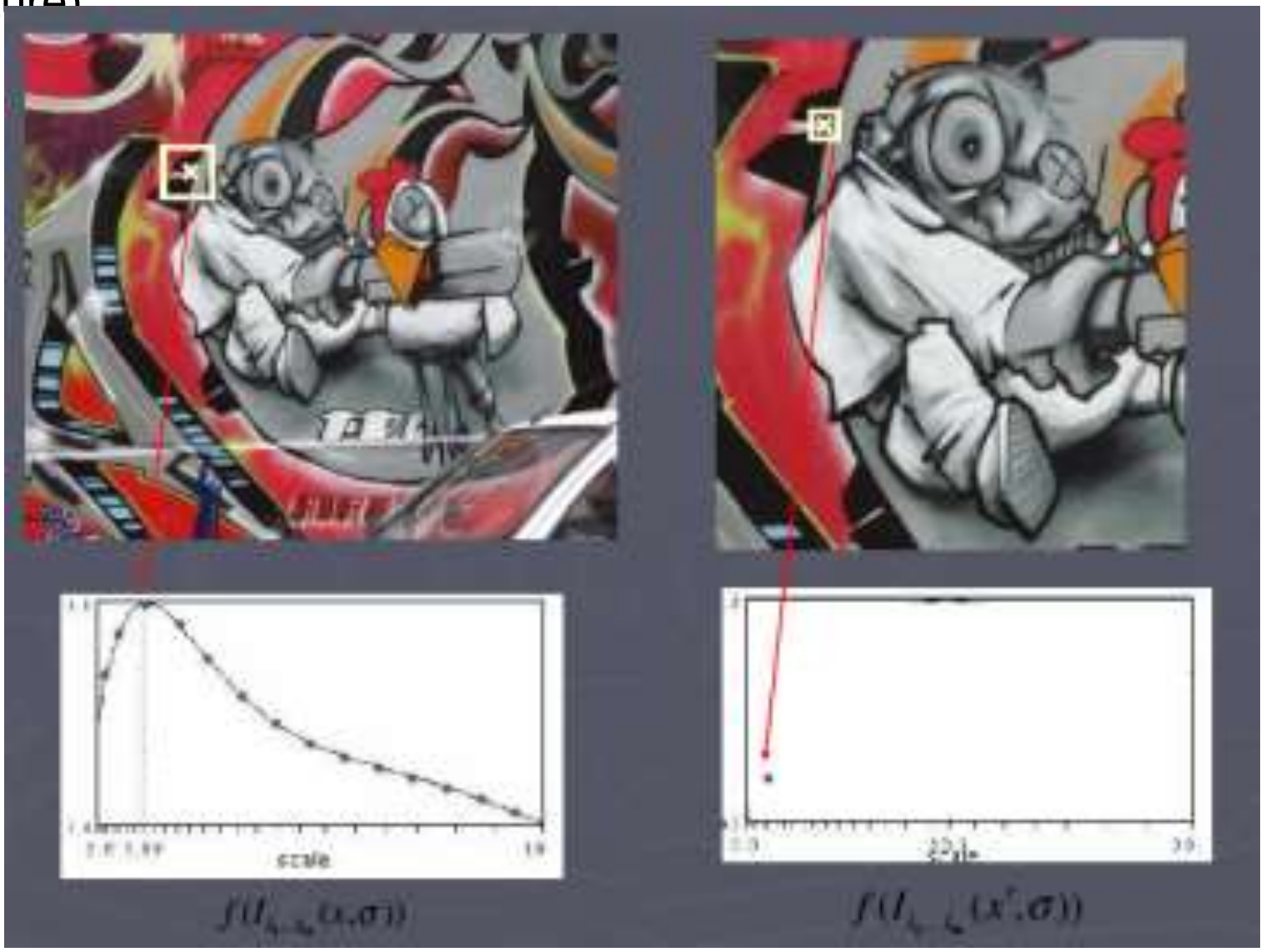
Automatic Scale Selection

Function responses for increasing scale (scale signature).



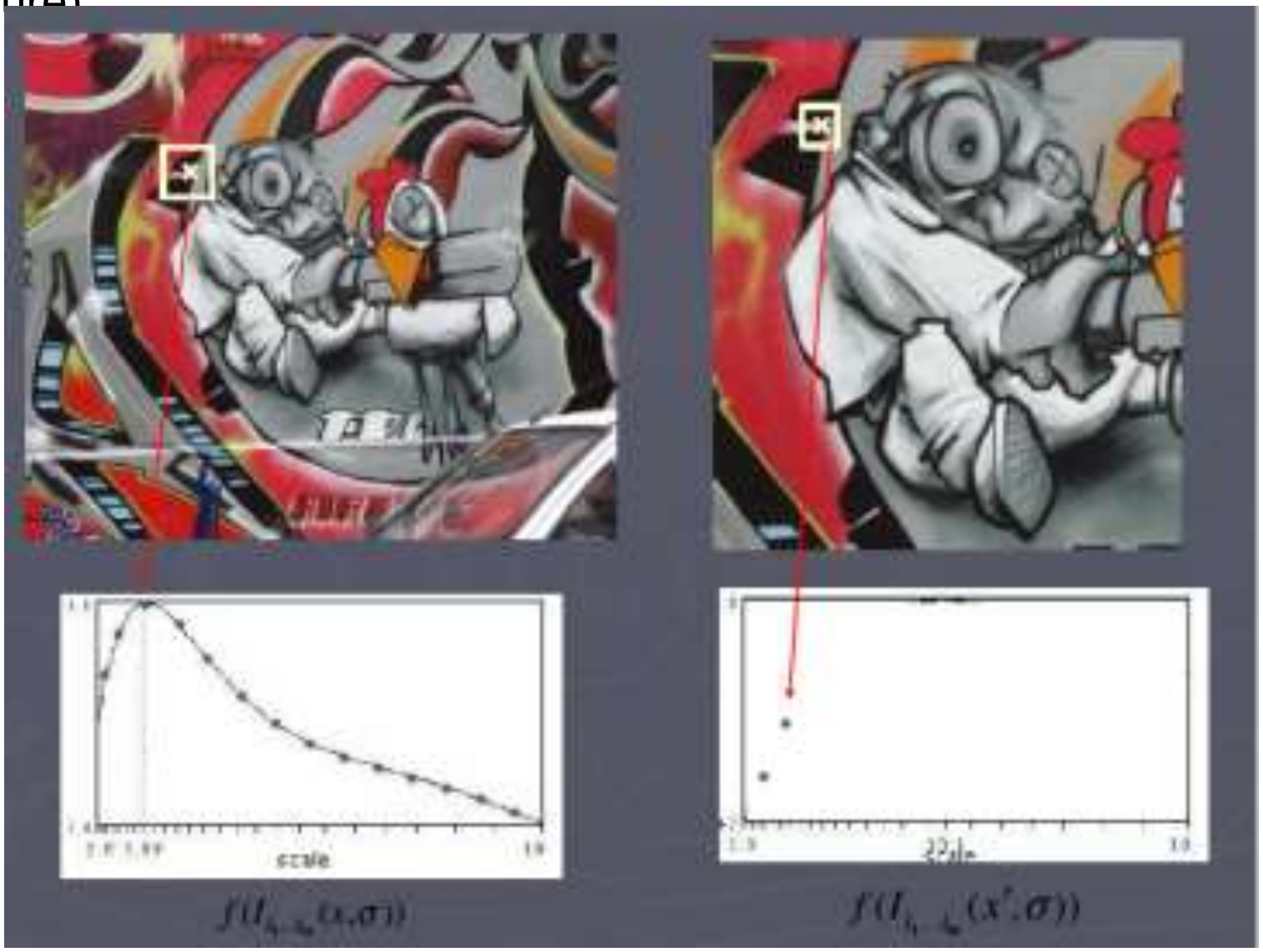
Automatic Scale Selection

Function responses for increasing scale (scale signature)



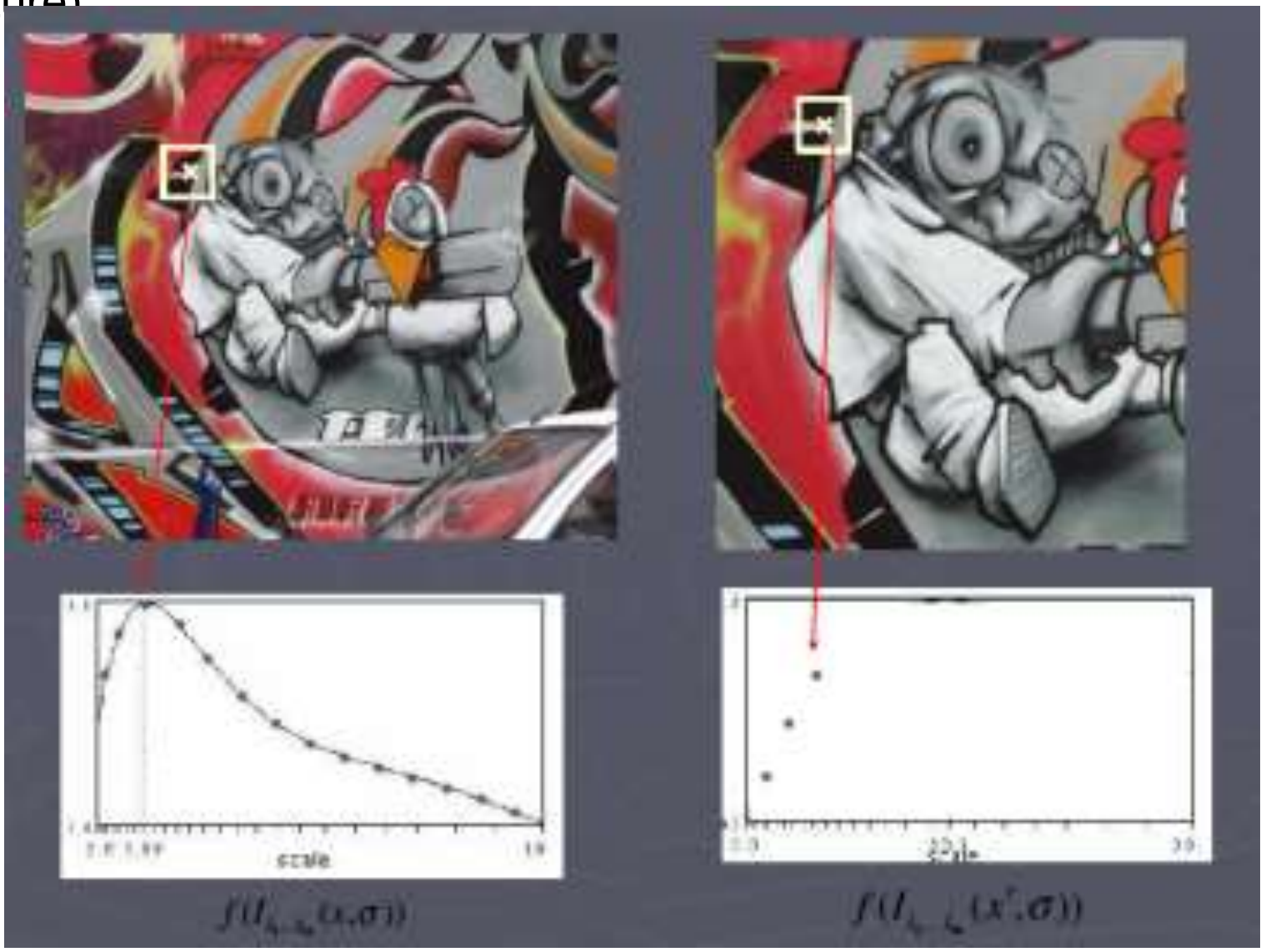
Automatic Scale Selection

Function responses for increasing scale (scale signature)



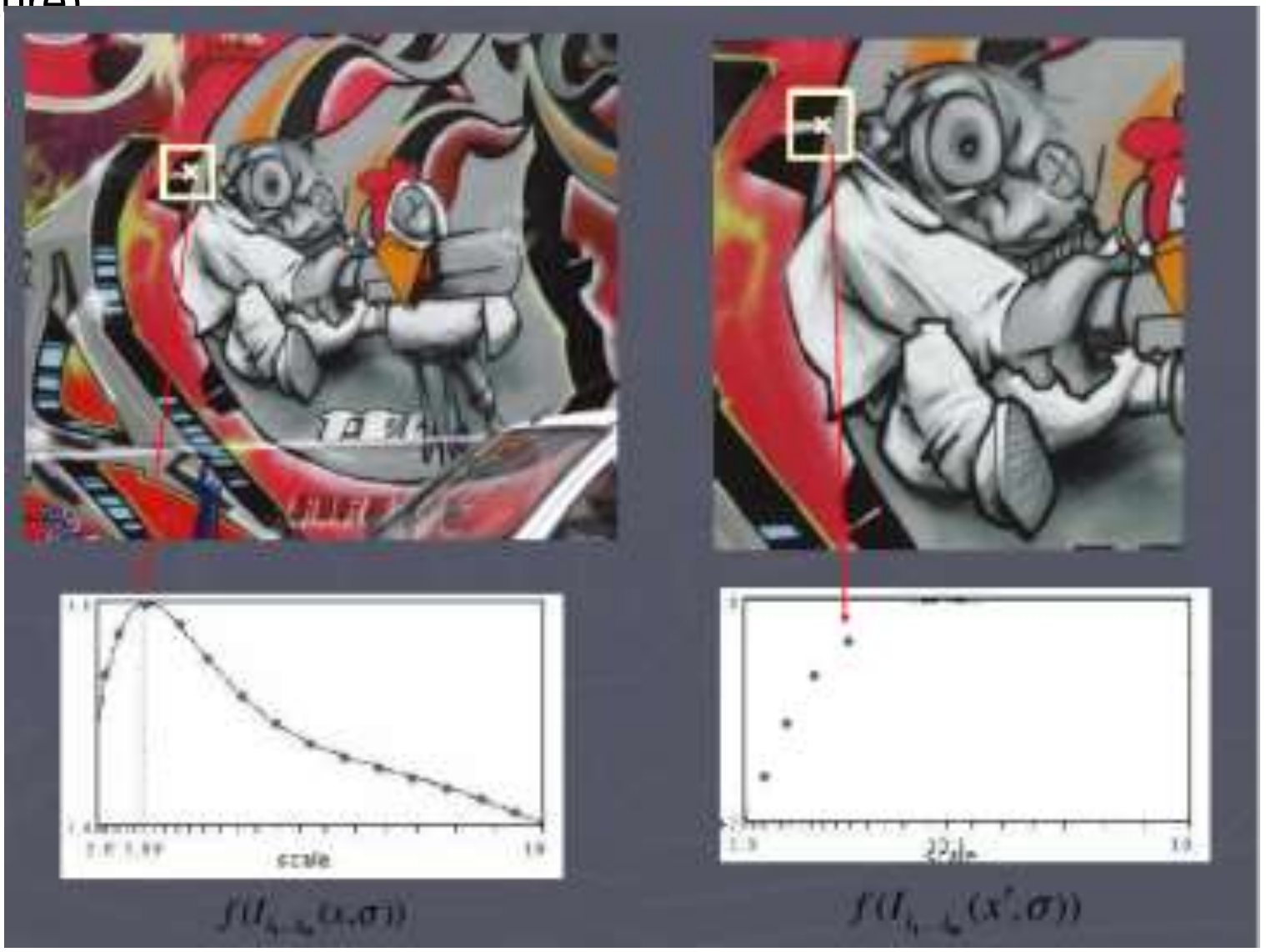
Automatic Scale Selection

Function responses for increasing scale (scale signature)



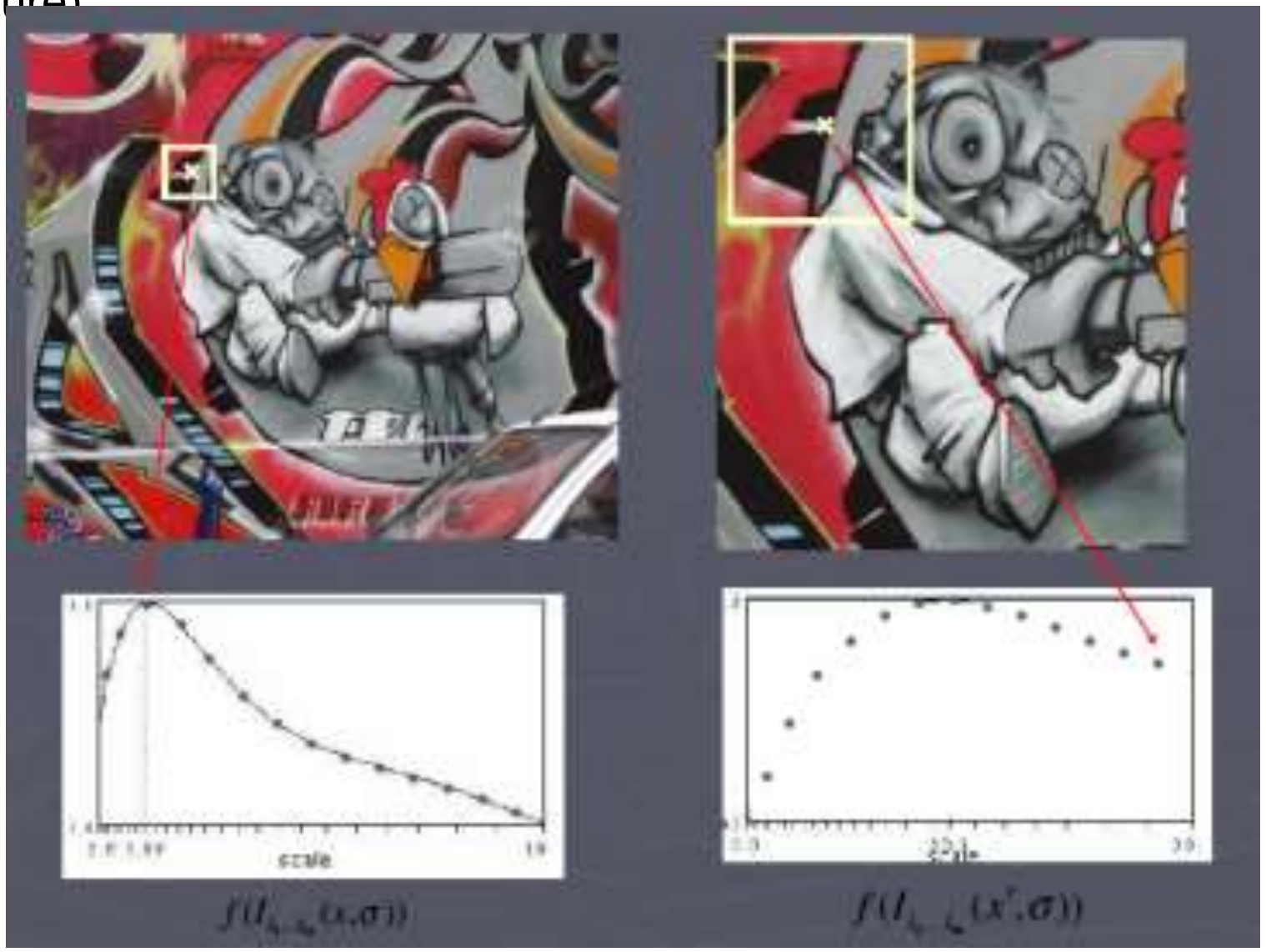
Automatic Scale Selection

Function responses for increasing scale (scale signature)



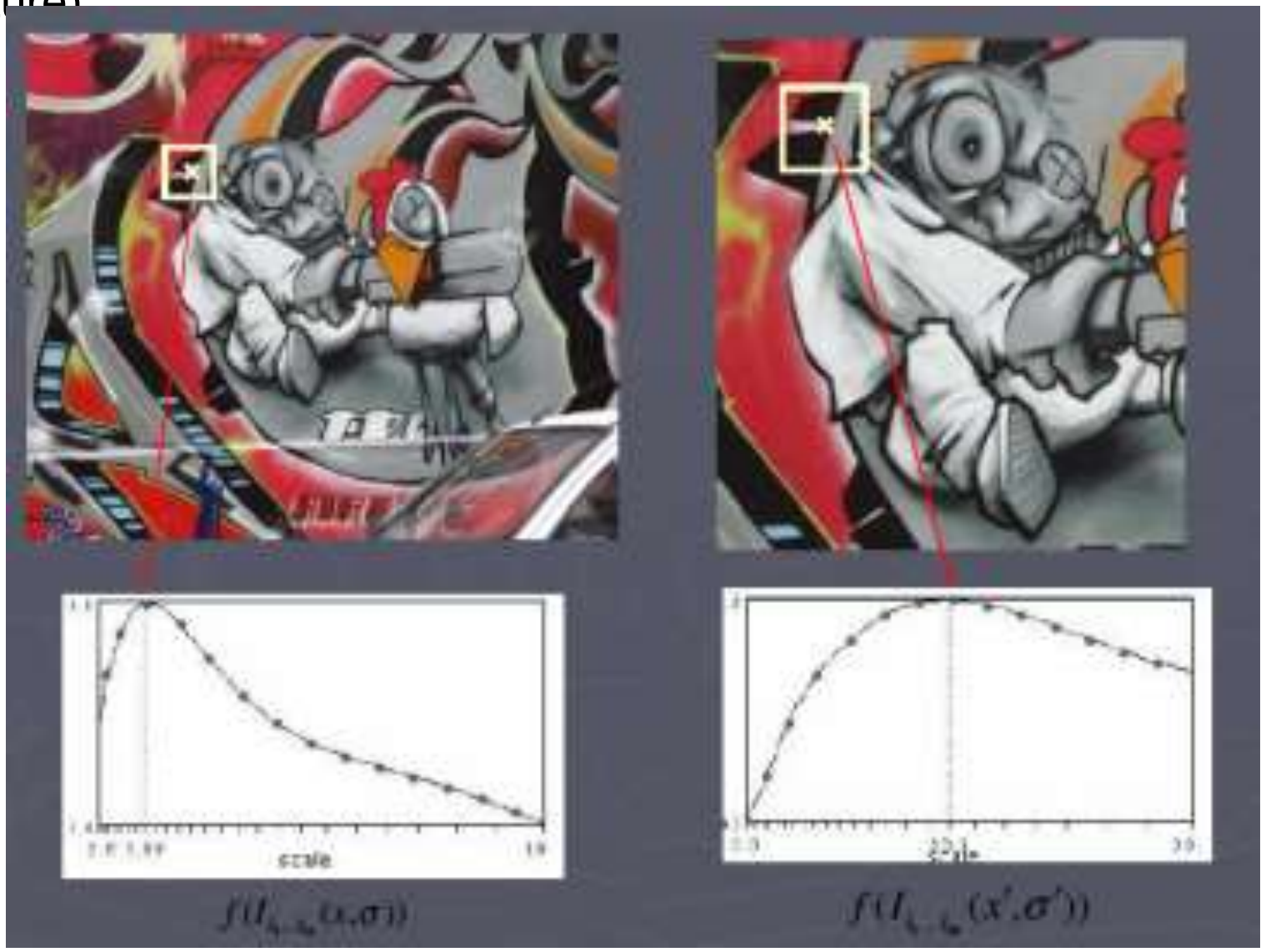
Automatic Scale Selection

Function responses for increasing scale (scale signature)



Automatic Scale Selection

Function responses for increasing scale (scale signature)



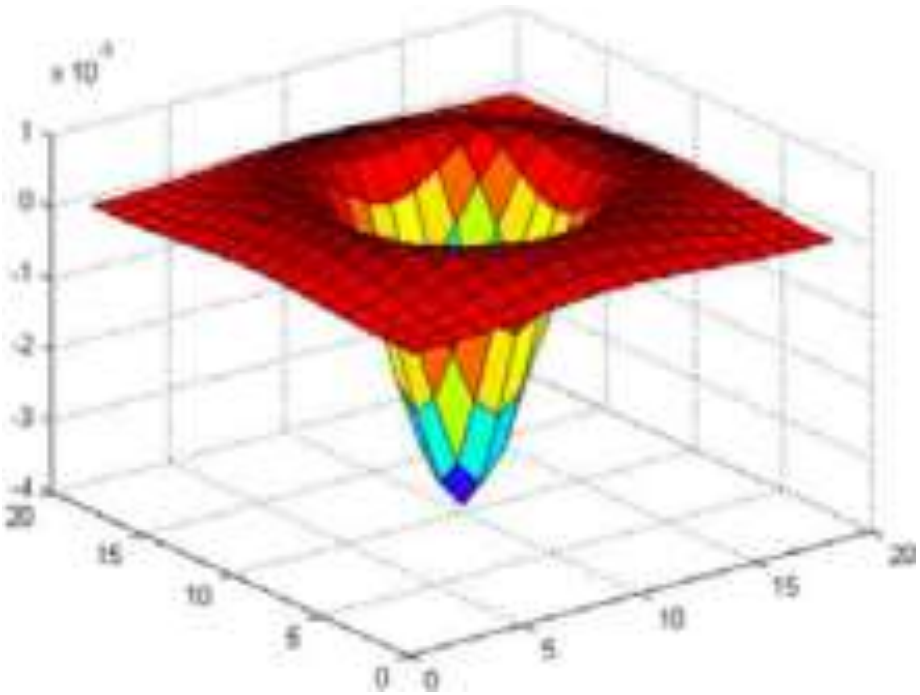
Blob Detection – Laplacian of Gaussian

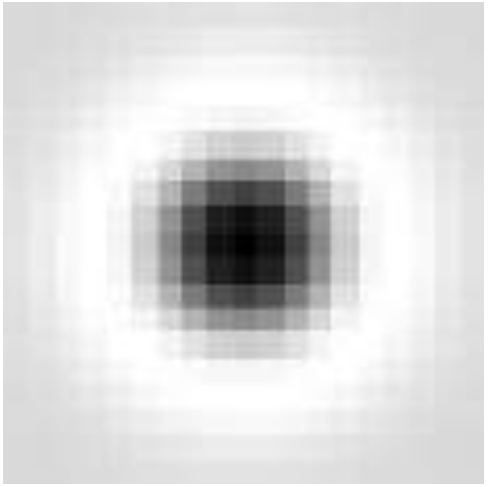
- Laplacian of Gaussian: We mentioned it for edge detection

$$\nabla^2 g(x, y, \sigma) = \frac{\partial^2 g(x, y, \sigma)}{\partial x^2} + \frac{\partial^2 g(x, y, \sigma)}{\partial y^2},$$

where g is a Gaussian

- It is a circularly symmetric operator (finds difference in all directions)
- It can be used for 2D blob detection





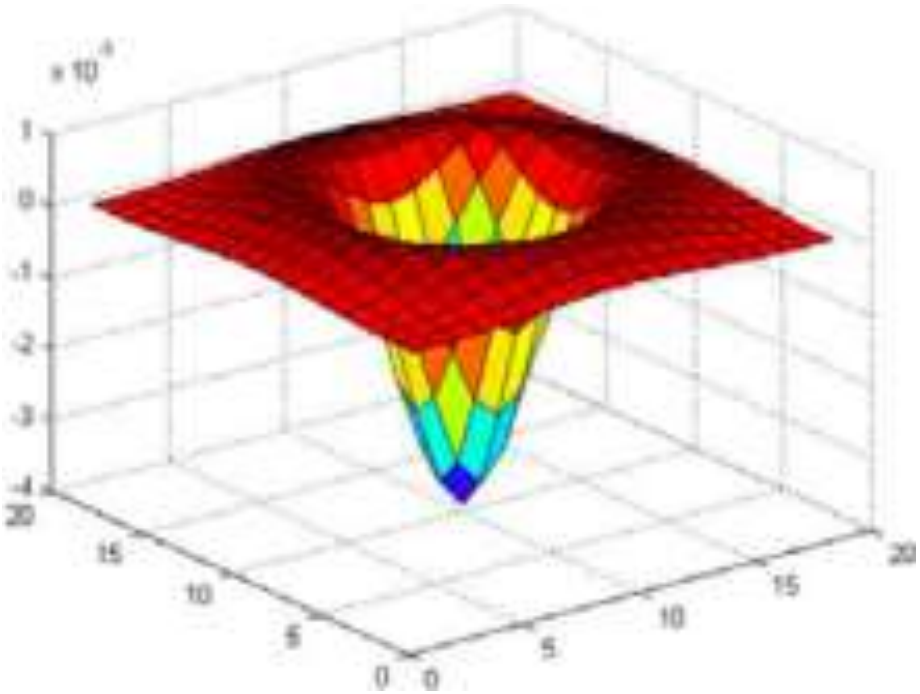
[Source: R. Gaussman, credit: D. Lowe]

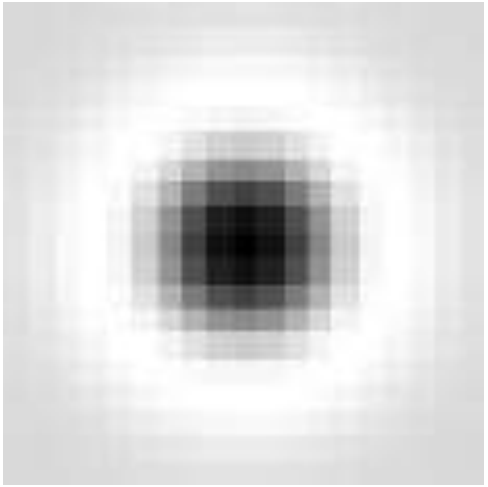
Blob Detection – Laplacian of Gaussian

- Laplacian of Gaussian: We mentioned it for edge detection

$$\mathrm{LoG}(x, y; \sigma) = \frac{1}{2\pi\sigma^4}\left(\frac{x^2+y^2-2\sigma^2}{\sigma^2}\right)\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$$

- It is a circularly symmetric operator (finds difference in all
- directions) It can be used for 2D blob detection

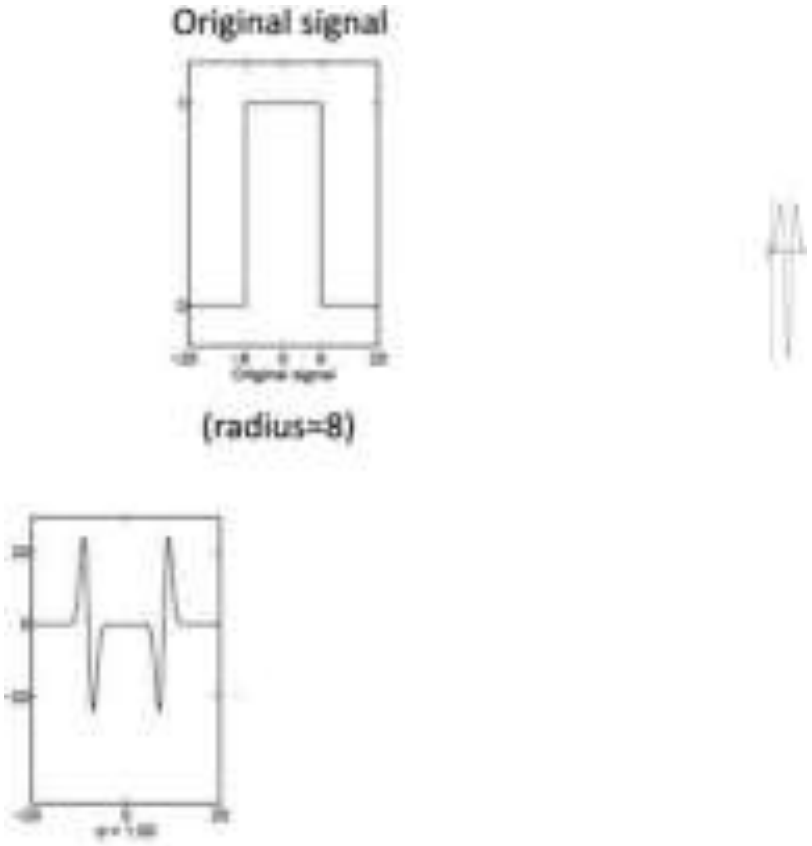




[Source: D. Forsyth, credit: D. Hoiem]

Blob Detection – Laplacian of Gaussian

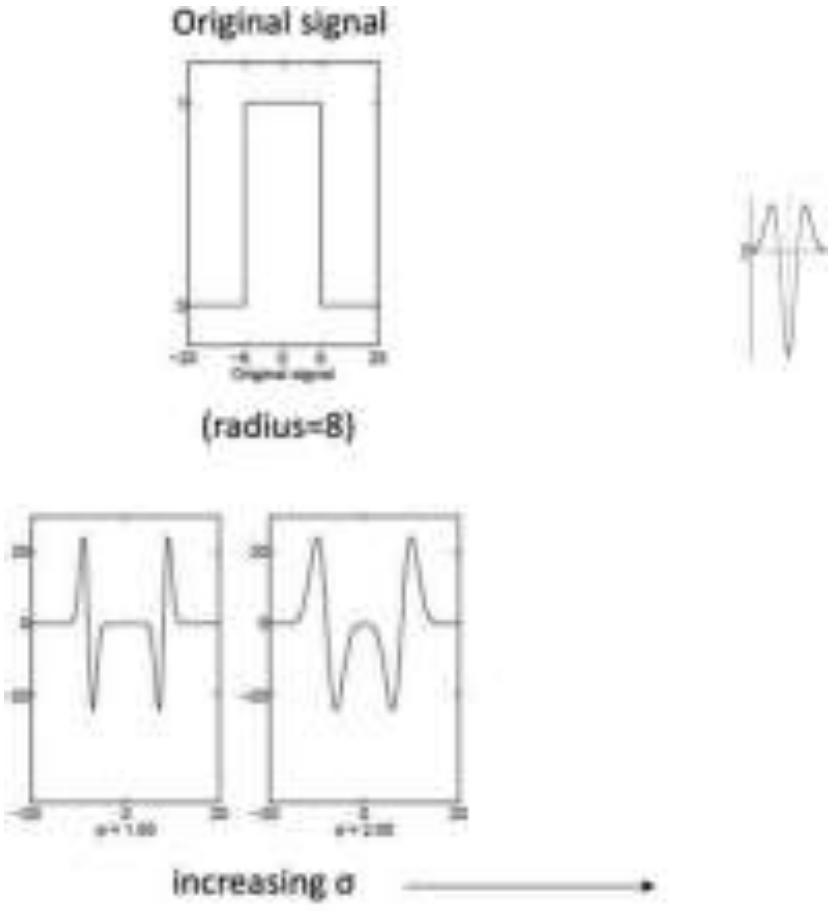
- It can be used for 2D blob detection! How?



[Source: F. Flores-Mangas]

Blob Detection – Laplacian of Gaussian

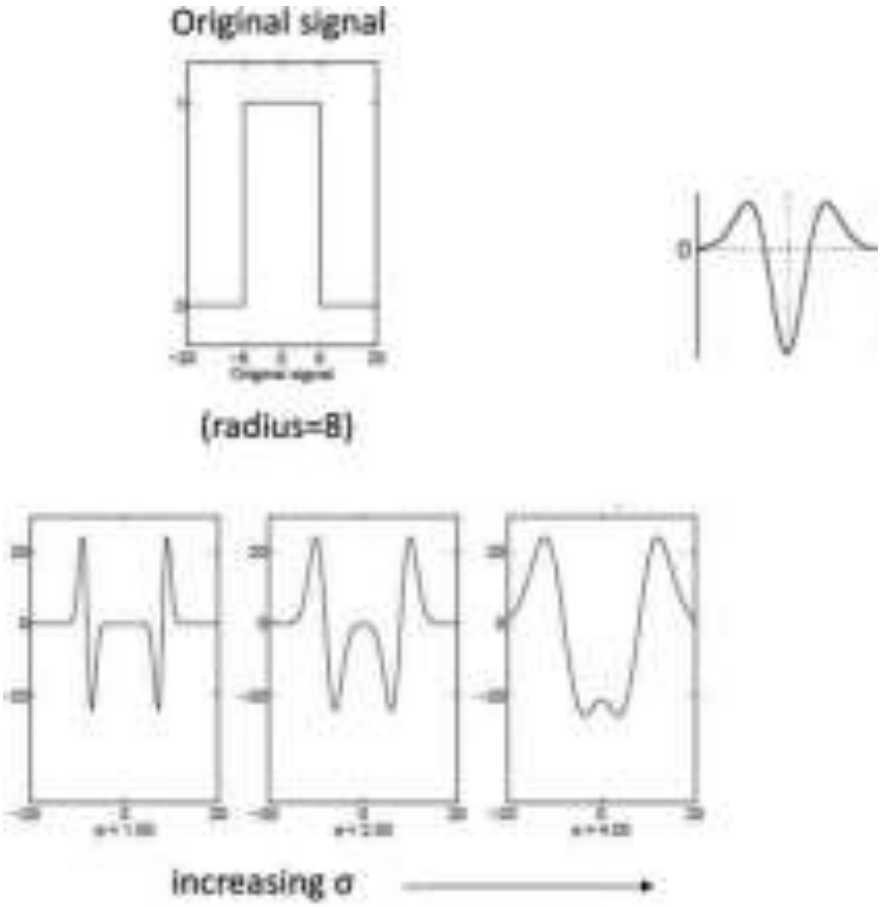
- It can be used for 2D blob detection!
  How?



[Source: F. Flores-Mangas]

Blob Detection – Laplacian of Gaussian

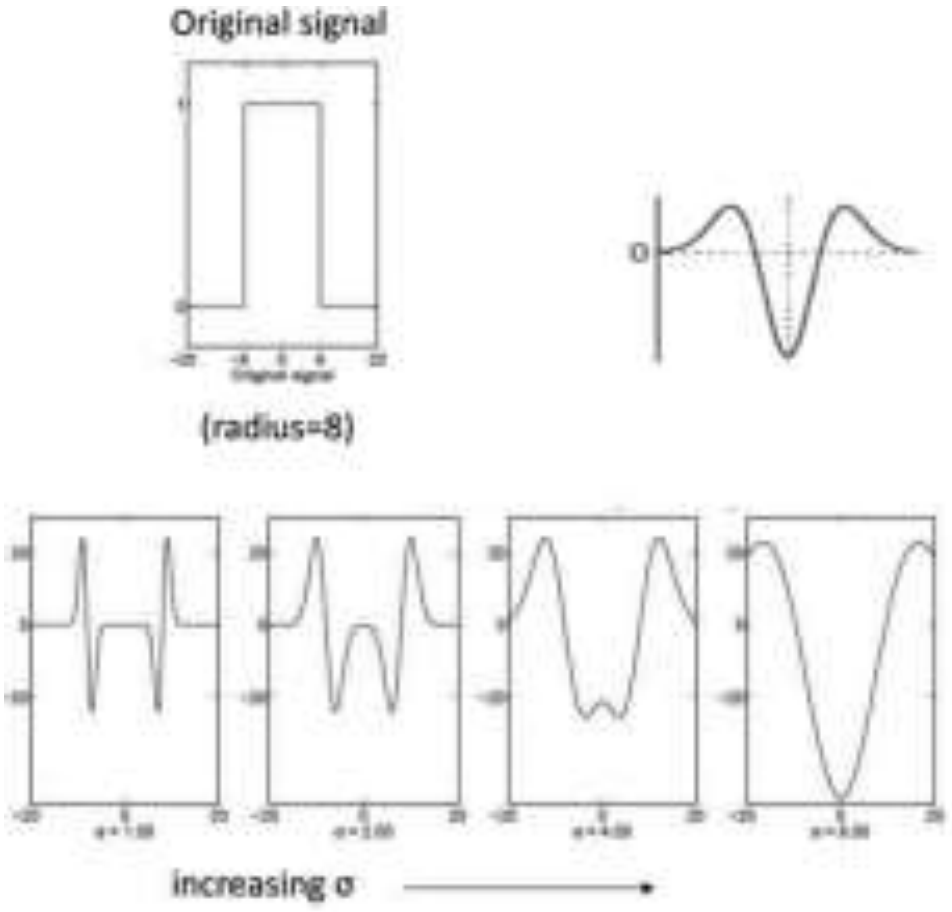
- It can be used for 2D blob detection!
  How?



[Source: F. Flores-Mangas]

Blob Detection – Laplacian of Gaussian

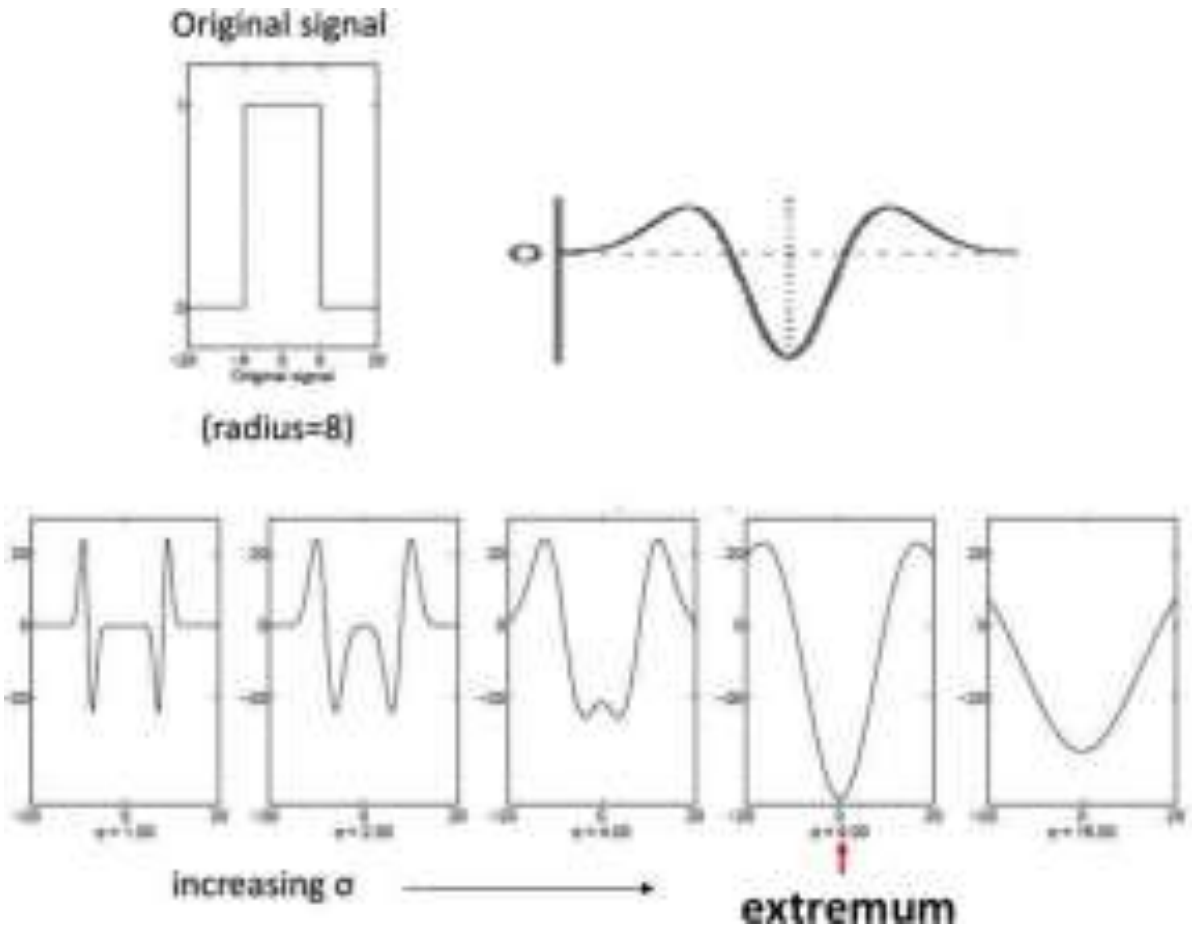
- It can be used for 2D blob detection!
  How?



[Source: F. Flores-Mangas]

Blob Detection – Laplacian of Gaussian

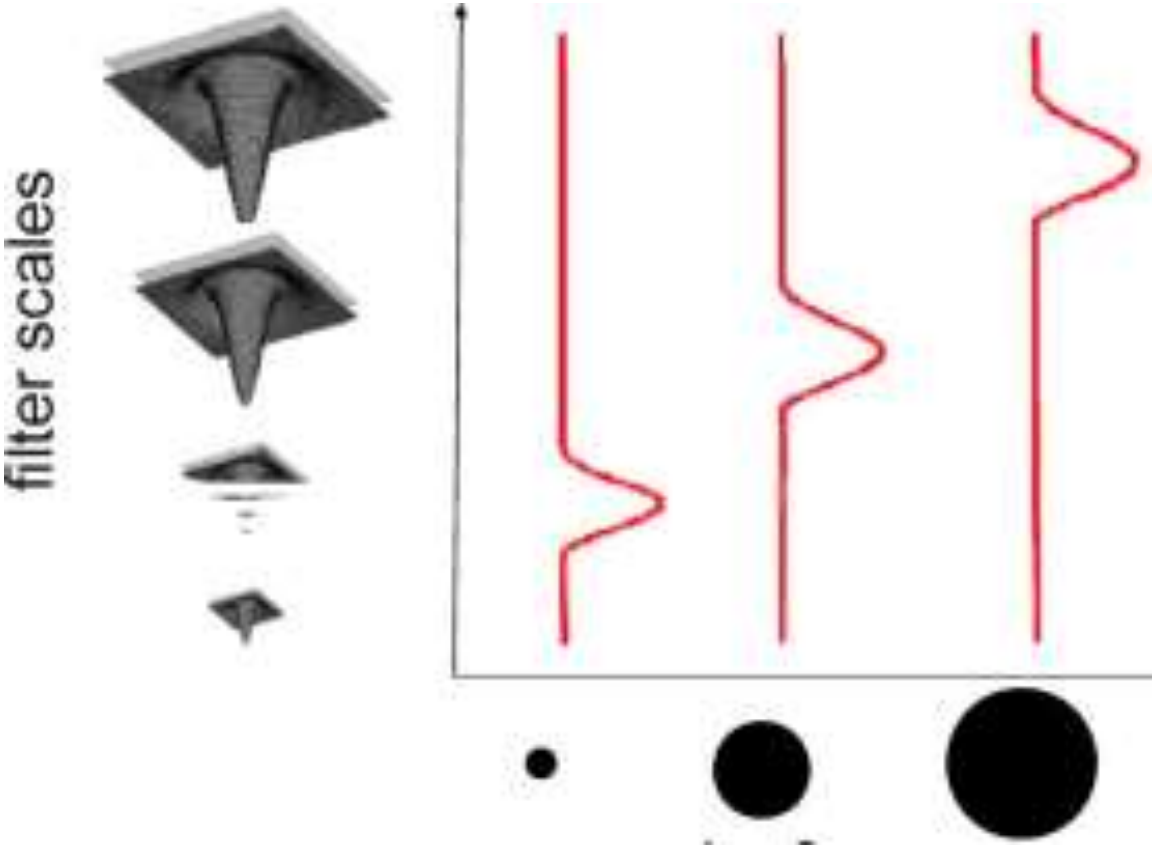
- It can be used for 2D blob detection!
  How?



[Source: F. Flores-Mangas]

Blob Detection in 2D: Scale Selection

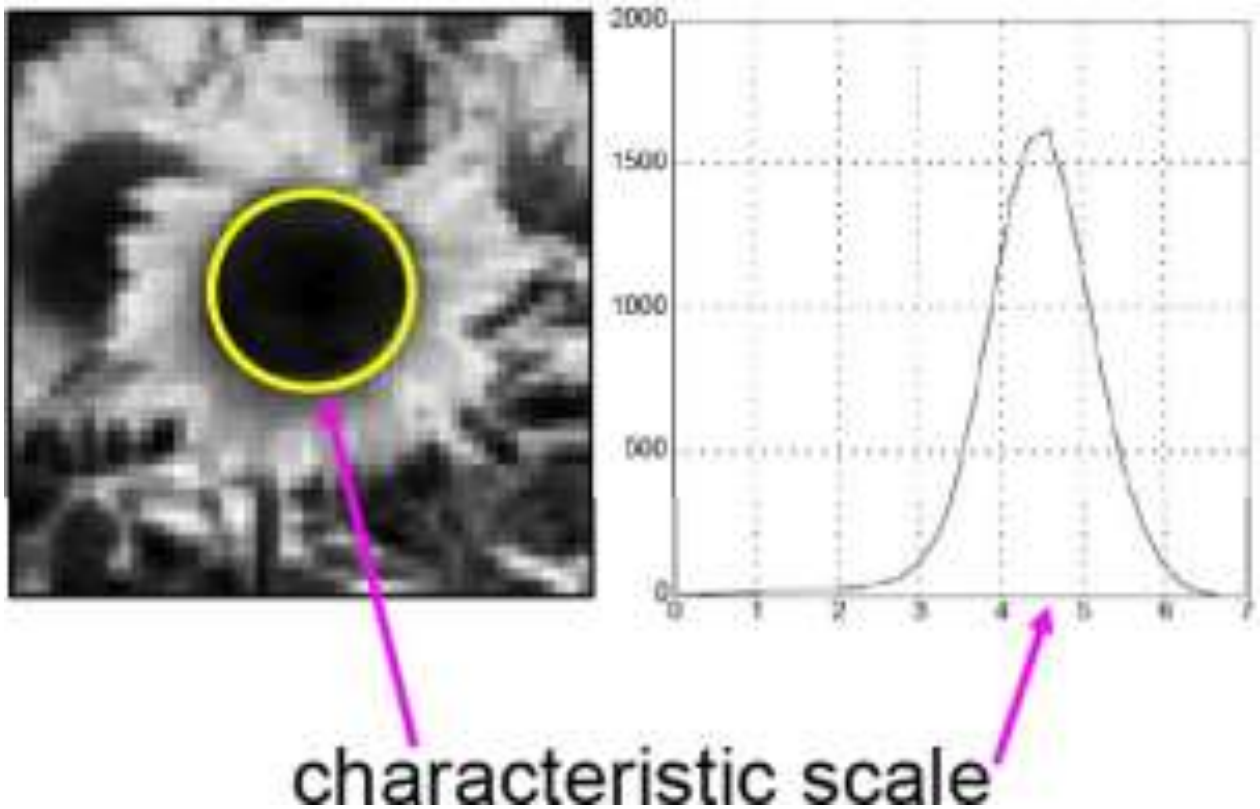
Laplacian of Gaussian = blob detector



[Source: B. Leibe, slide credit: R. Urtasun]

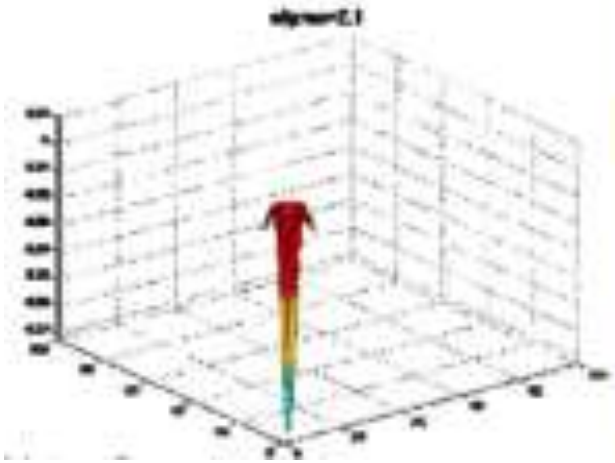
Characteristic Scale

- We define the **characteristic scale** as the scale that produces peak (minimum or maximum) of the Laplacian response



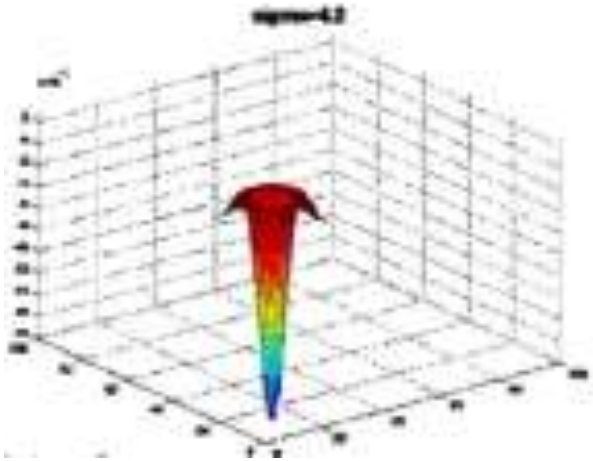
[Source: S. Lazebnik]

Example



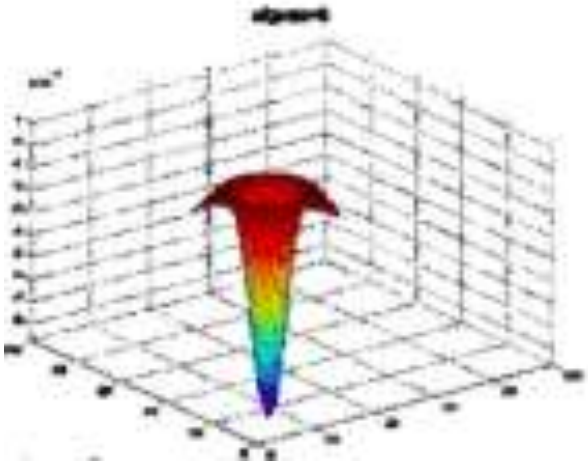
[Source: K. Grauman]

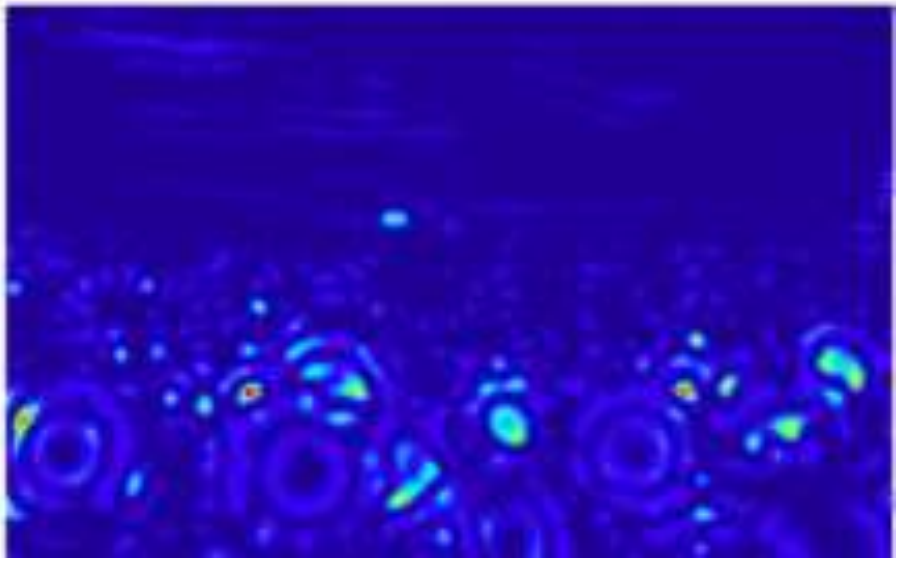
Example



[Source: K. Grauman]

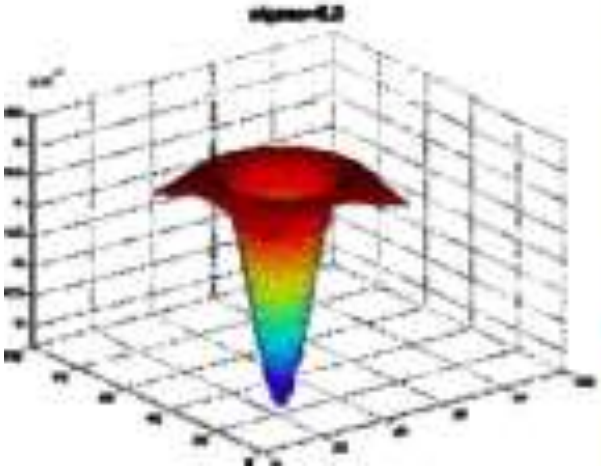
Example

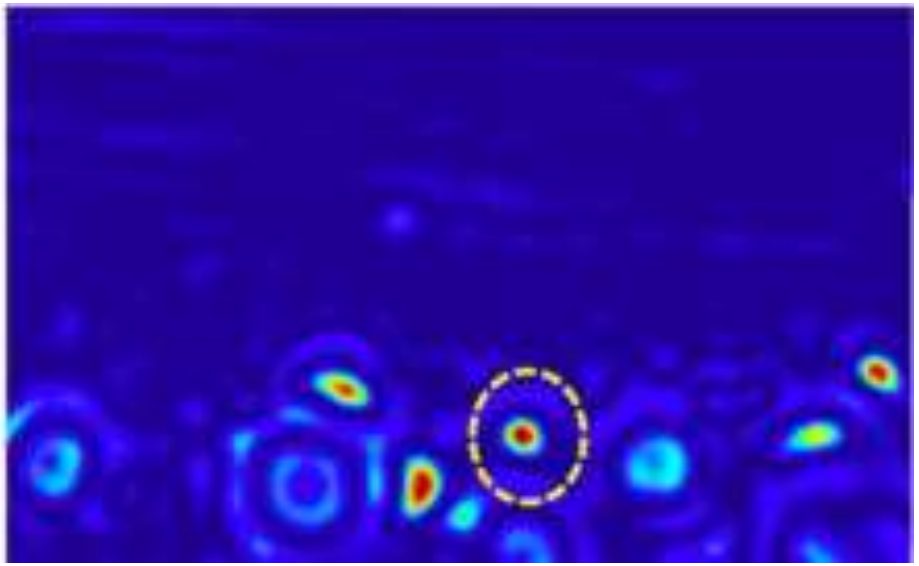




[Source: K. Grauman]

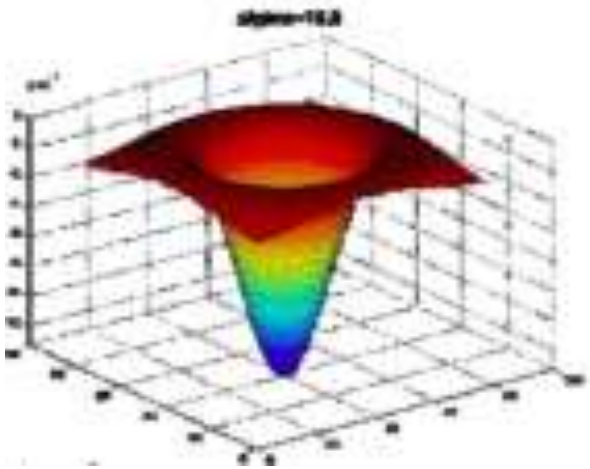
Example

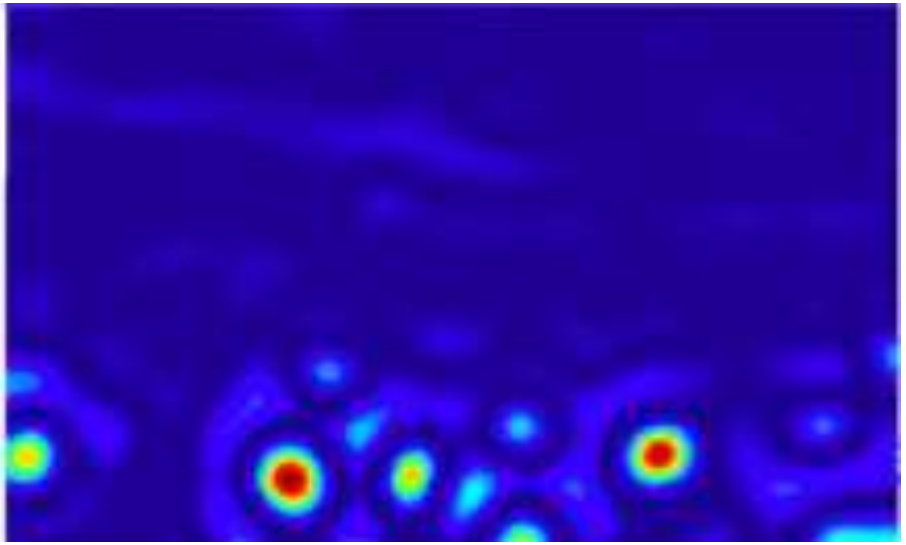




[Source: K. Grauman]

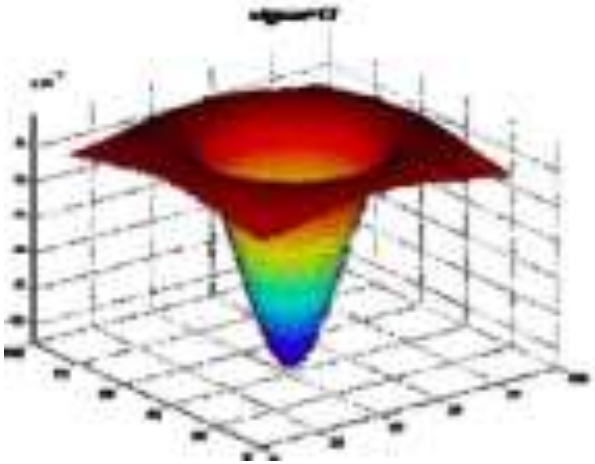
Example

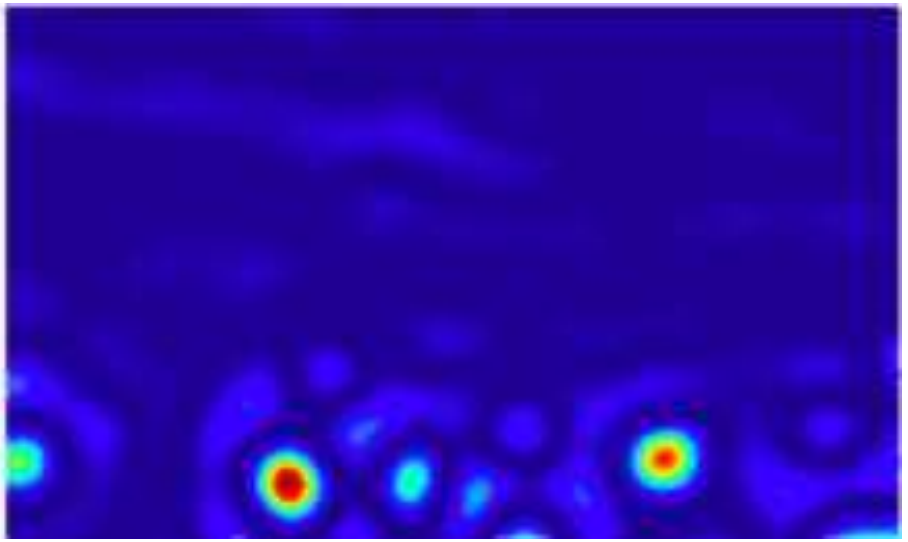




[Source: K. Grauman]

Example





[Source: K. Grauman]

Efficient implementation

Courtesy: Forsyth and Ponce

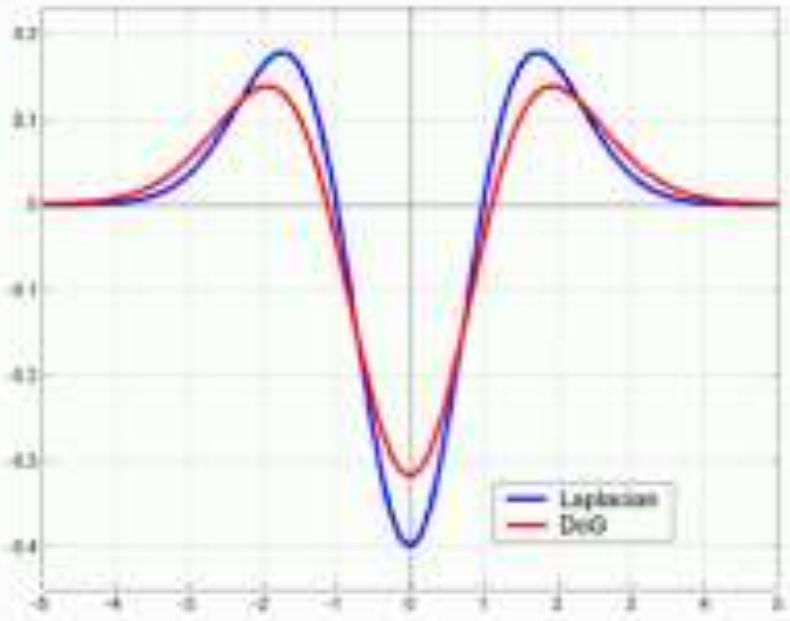
Approximating the Laplacian with a difference of Gaussians:

$$L = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right)$$

(Laplacian)

$$DoG = G(x, y, k\sigma) - G(x, y, \sigma)$$

(Difference of Gaussians)



Derivation based on heat diffusion equation. For more details see Sec 3, pgs 5-6 of David Lowe, IJCV 2004 paper: https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf

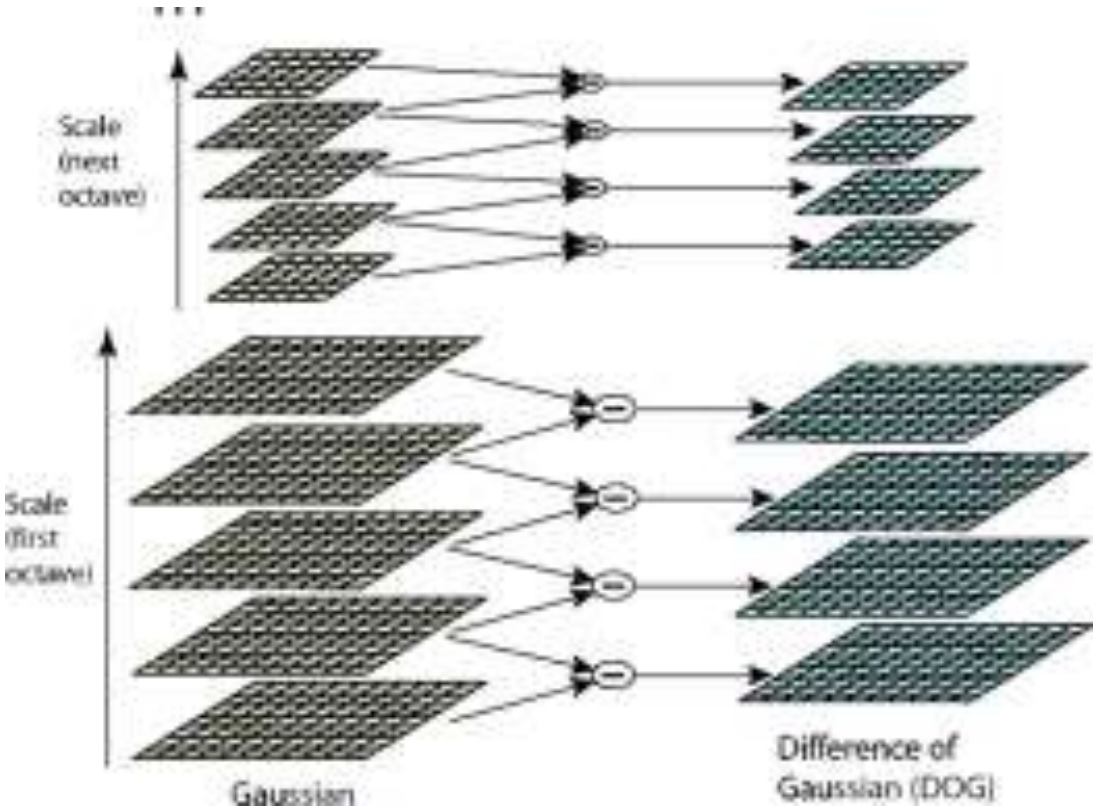
Scale Invariant Feature Transform (SIFT)

Steps in SIFT

- Step 1: Approximate keypoint location
- Step 2: Refining keypoint location
- Step 3: Assigning orientations
- Step 4: Descriptors for each keypoint

Step 1: Approximate keypoint

Down-
samplin
g



$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

Octave = doubling of σ_0. Within an octave, the adjacent scales differ by a constant factor *k*. If an octave contains *s+1* images, then $k = 2^{(1/(s))}$. The first image has scale σ_0, the second image has scale $k\sigma_0$, the third image has scale $k^2\sigma_0$, and the last image has scale $k^s\sigma_0$. Such a sequence of images convolved with Gaussians of increasing σ constitute a so-called **scale space**.

Scale = 1

Scale = 4

Scale = 16

Scale = 64

Scale = 256

http://en.wikipedia.org/wiki/Scale_space

Step 1: Approximate keypoint

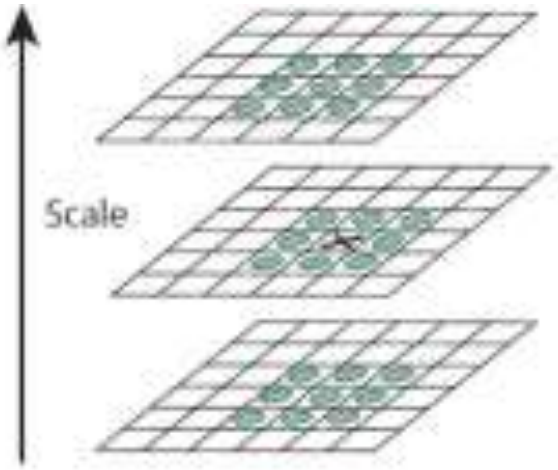


Image taken from D. Lowe, “Distinctive Image Features from Scale-Invariant Points”, IJCV 2004

Figure 2: Maxima and minima of the difference-of-Gaussian images are detected by comparing a pixel (marked with X) to its 26 neighbors in 3x3 regions at the current and adjacent scales (marked with circles).

The keypoints are maxima or minima in the “scale-space-pyramid”,
i.e. the stack of DoG images. Hereby, you get both the location as well as the scale of the keypoint.

Initial detection of keypoints

http://upload.wikimedia.org/wikipedia/commons/4/44/Sift_keypoints_filtering.jpg

Step 2: Refining keypoint location

- The keypoint location and scale is discrete – we can interpolate for greater accuracy.
- For this, we express the DoG function in a small 3D neighborhood around a keypoint (x_i, y_i, σ_i) by a second-order Taylor-series:

$$D(x, y, \sigma) = D(x_i, y_i, \sigma_i) + \boxed{\left(\frac{\partial D(x, y, \sigma)}{\partial(x, y, \sigma)}\right)^T \Delta} + \frac{1}{2}\Delta^T \boxed{\frac{\partial^2 D(x, y, \sigma)}{\partial(x, y, \sigma)^2}} \Delta;$$

$$\Delta = \begin{pmatrix} x - x_i \\ y - y_i \\ \sigma - \sigma_i \end{pmatrix}$$

Gradient vector evaluated digitally at the keypoint

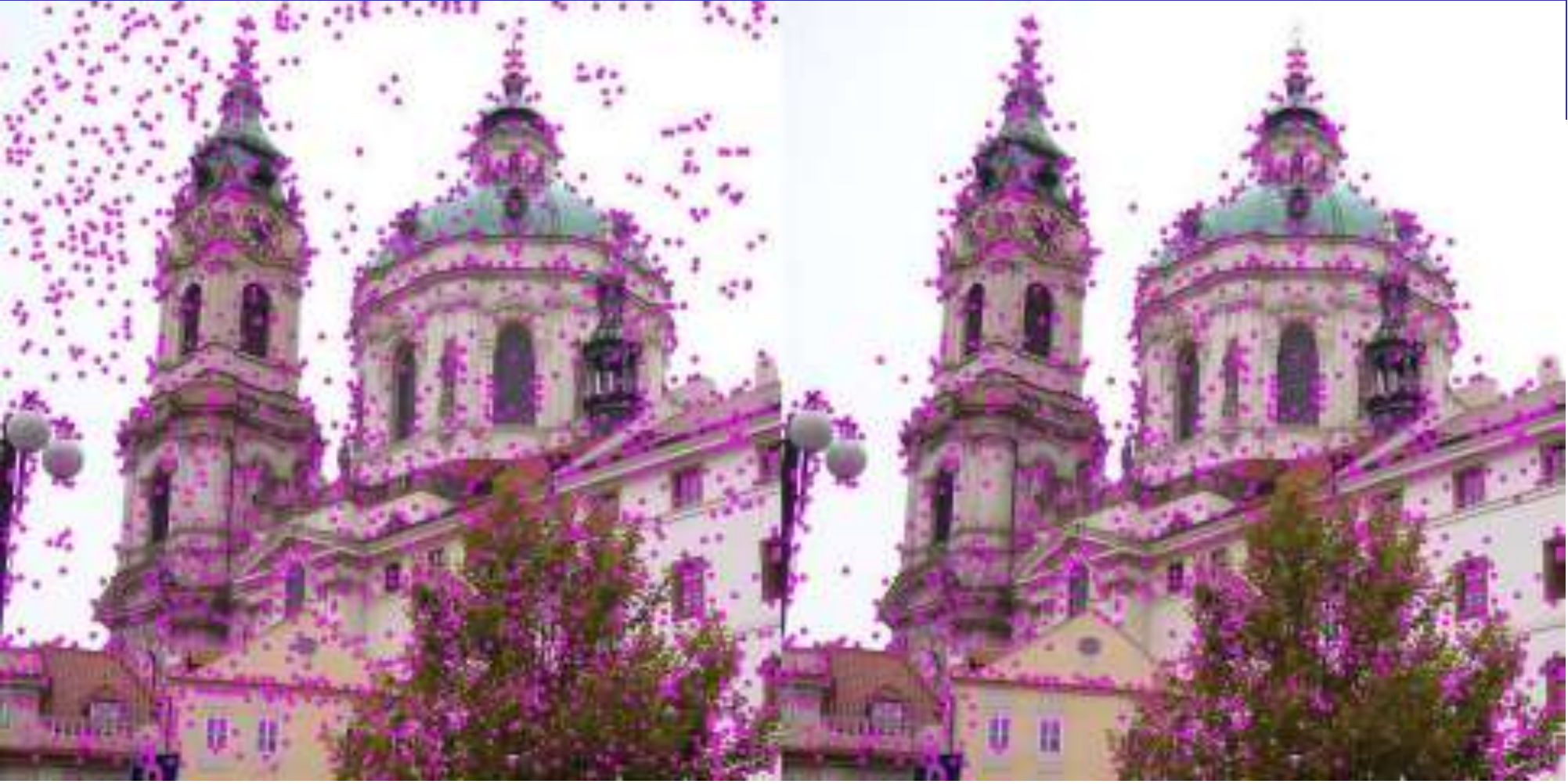
3 x 3 Hessian matrix evaluated digitally at the keypoint

Step 2: Refining keypoint location

- To find an extremum of the DoG values in this neighborhood, set the derivative of D(.) to 0. This gives us:

$$\Delta = -1 \left(\frac{\partial^2 D(x, y, \sigma)}{\partial(x, y, \sigma)^2} \right)^{-1}_{\substack{x = x_i, \\ y = y_i, \\ \sigma = \sigma_i}} \left(\frac{\partial D(x, y, \sigma)}{\partial(x, y, \sigma)} \right)_{\substack{x = x_i, \\ y = y_i, \\ \sigma = \sigma_i}}$$

- The keypoint location is updated.
- All extrema with $|D_{extremal}| < 0.03$, are discarded as "weak extrema" or "low contrast points".



Removal of low-contrast keypoints

http://upload.wikimedia.org/wikipedia/commons/4/44/Sift_keypoints_filtering.jpg

Step 2: Refining keypoint location

- Some keypoints reside on edges, as edges always give a high response to a DoG filter.
- But edges should not be considered salient points (why?).
- So we discard points that lie on edges.
- In the case of KLT tracker, we saw how to detect points lying on salient edges using the structure tensor.

Step 2: Refining keypoint location

- The SIFT paper uses the 2nd derivative matrix (called the Hessian matrix):

$$\mathbf{H} = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$

- The eigenvalues of **H** give a lot of information about the local structure around the keypoint.
- In fact, the eigenvalues are the **maximal** and **minimal principal curvatures** of the surface D(x,y), i.e. of the DoG function, at that point.

http://en.wikipedia.org/wiki/File:Minimal_surface_curvature_planes-en.svg

Step 2: Refining keypoint location

- An edge will have **high maximal** curvature, but very **low minimal** curvature.
- A keypoint which is a corner (not an edge) will have **high maximal and minimal** curvature.
- The following can be regarded as an

$$\mathrm{Tr}(\mathbf{H}) = D_{xx} + D_{yy} = \alpha + \beta,$$

$$\mathrm{Det}(\mathbf{H}) = D_{xx}D_{yy} - (D_{xy})^2 = \alpha\beta.$$

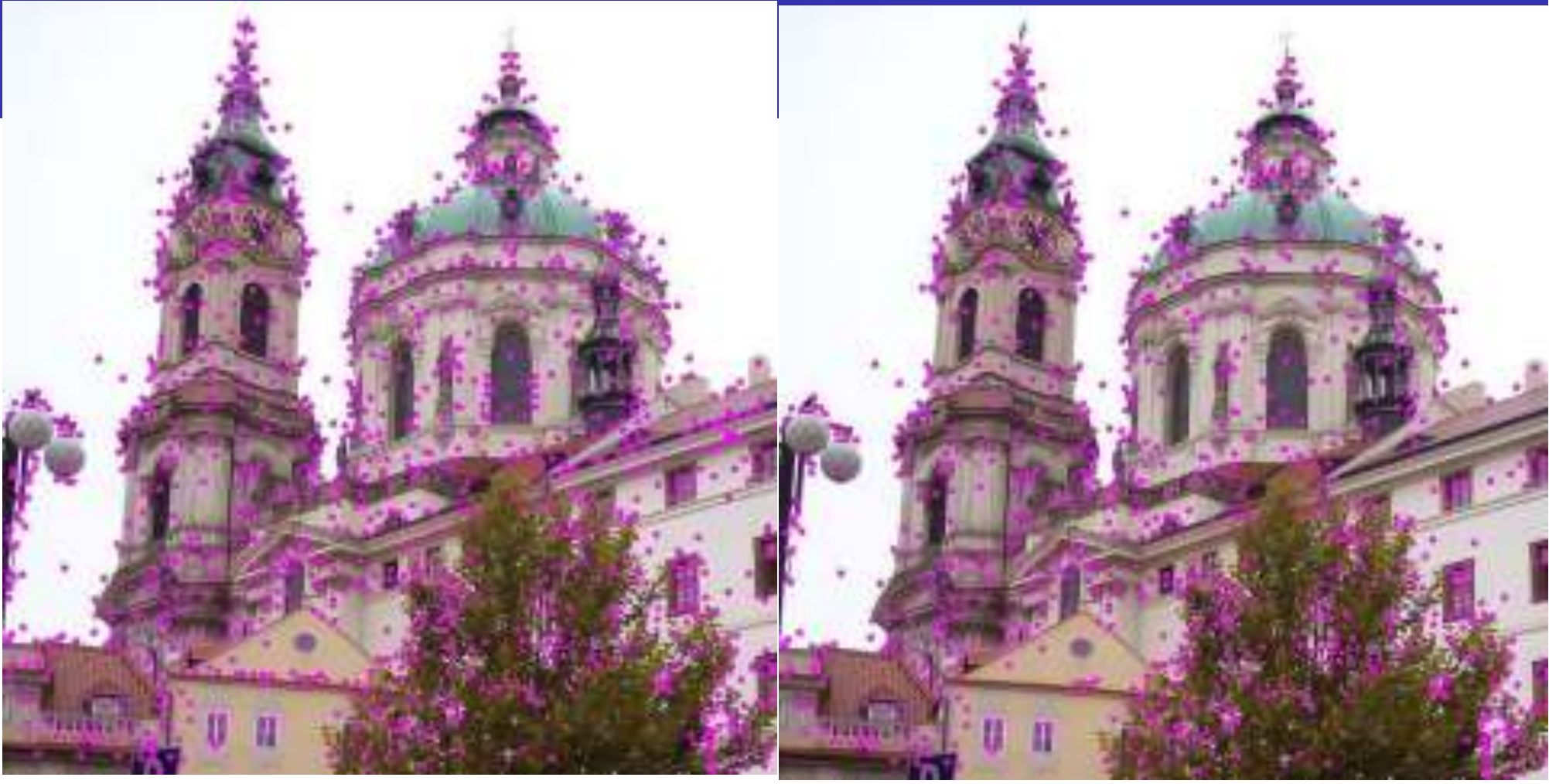
$$\boxed{\alpha = r\beta}$$

$$\boxed{\frac{\mathrm{Tr}(\mathbf{H})^2}{\mathrm{Det}(\mathbf{H})}} = \frac{(\alpha+\beta)^2}{\alpha\beta} = \frac{(r\beta+\beta)^2}{r\beta^2} = \frac{(r+1)^2}{r}$$

For a corner, should be less than a threshold (say 10).

For an edge, α >> β, leading to a large value of this measure.

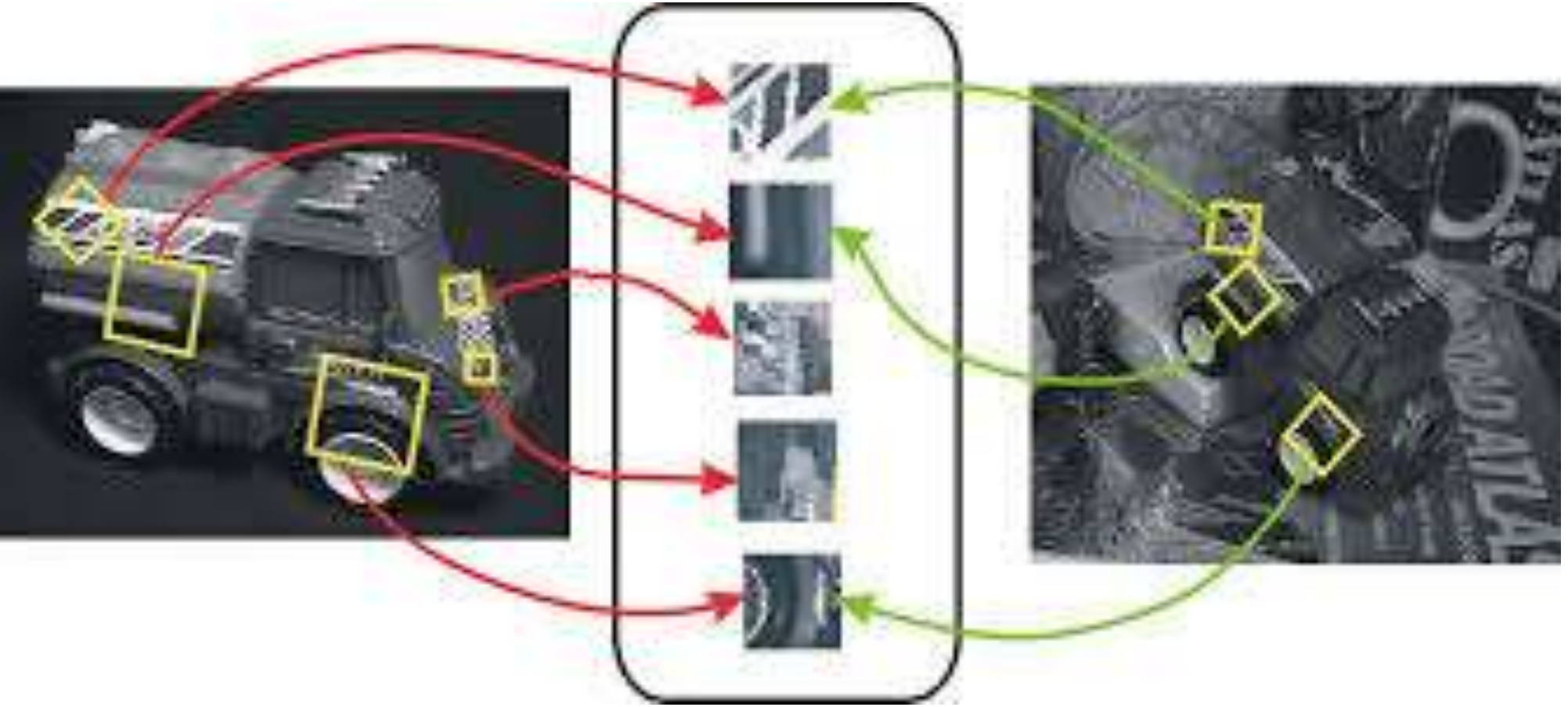
Why this measure instead of r? To save computations – we need not compute eigenvalues!



Removal of high-contrast keypoints residing on edges

http://upload.wikimedia.org/wikipedia/commons/4/44/Sift_keypoints_filtering.jpg

Step 3: Assigning orientations



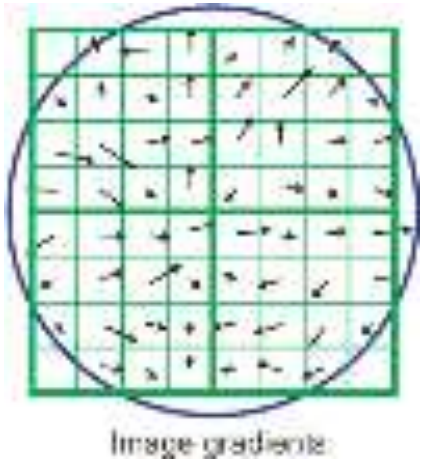
Step 3: Assigning orientations

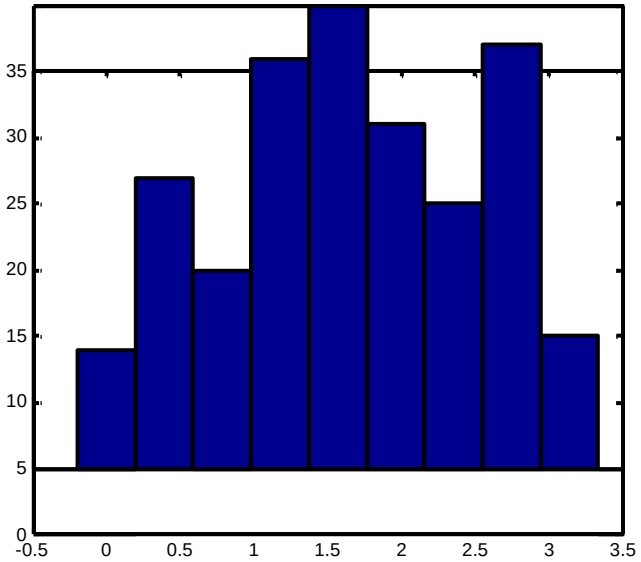
- Compute the gradient magnitudes and orientations in a small window around the keypoint – at the appropriate scale.

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

$$m(x, y) = \sqrt{(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2}$$

$$\theta(x, y) = \tan^{-1}((L(x, y+1) - L(x, y-1))/(L(x+1, y) - L(x-1, y)))$$

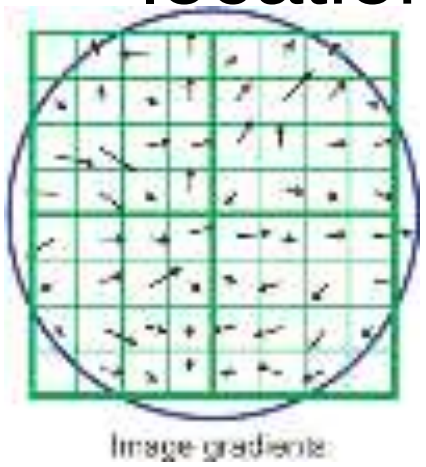


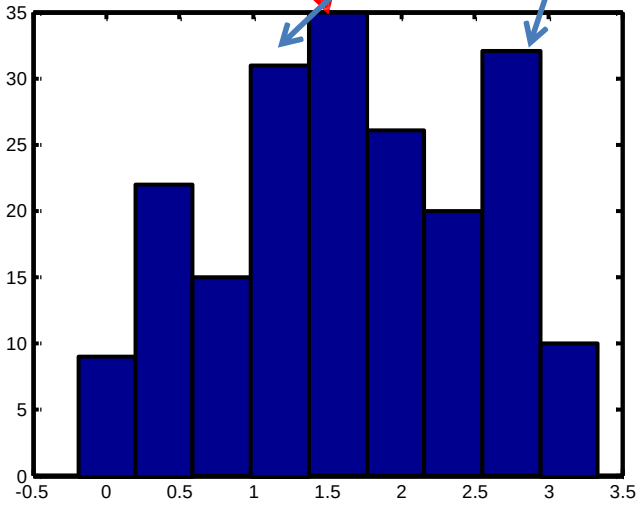


Histogram of gradient orientation – the bin-counts are weighted by gradient magnitudes and a Gaussian weighting function. Usually, 36 bins are chosen for the orientation.

Step 3: Assigning orientations

- Assign the **dominant orientation** as the orientation of the keypoint.
- In case of multiple peaks or histogram entries more than 0.8 x peak, create a **separate** descriptor for ***each*** orientation (they will all have the same scale and location).





Histogram of gradient orientation – the bin-counts are weighted by gradient magnitudes and a Gaussian weighting function. Usually, 36 bins are chosen for the orientation.

Step 4: Descriptors for each

- Consider a small region around the keypoint. Divide it into *n* x *n* cells (usually *n* = 2). Each cell is of size 4 x 4.
- Build a **gradient orientation histogram** in each cell. Each histogram entry is weighted by the *gradient magnitude* and a *Gaussian weighting function* with σ = 0.5 times window width.
- **Sort** each gradient orientation histogram bearing in mind the dominant orientation of the keypoint (assigned in step 3).

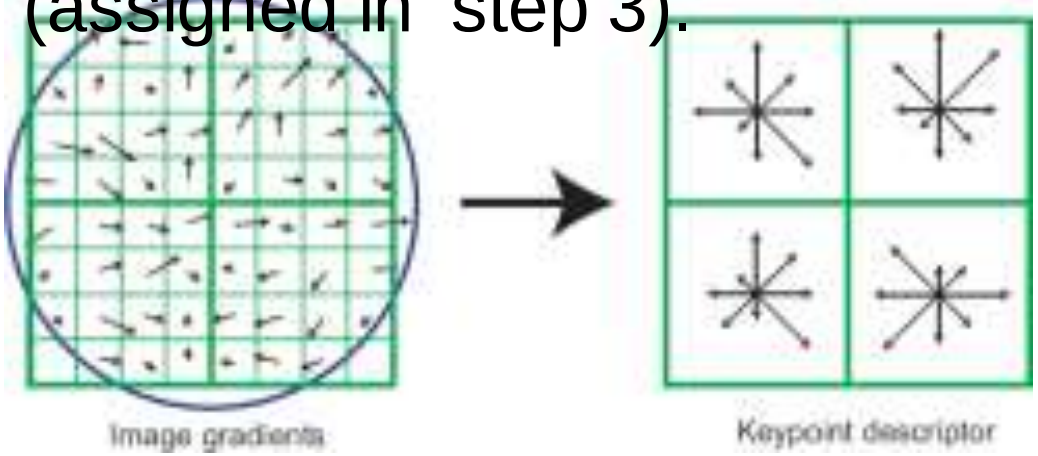


Image taken from D. Lowe, "Distinctive Image Features from Scale-Invariant Points", IJCV 2004

Step 4: Descriptors for each

- We now have a descriptor of size rn^2 if there are r bins in the orientation histogram.
- Typical case used in the SIFT paper: $r = 8$, $n = 4$, so length of each descriptor is 128.
- The descriptor is invariant to rotations due to the sorting.

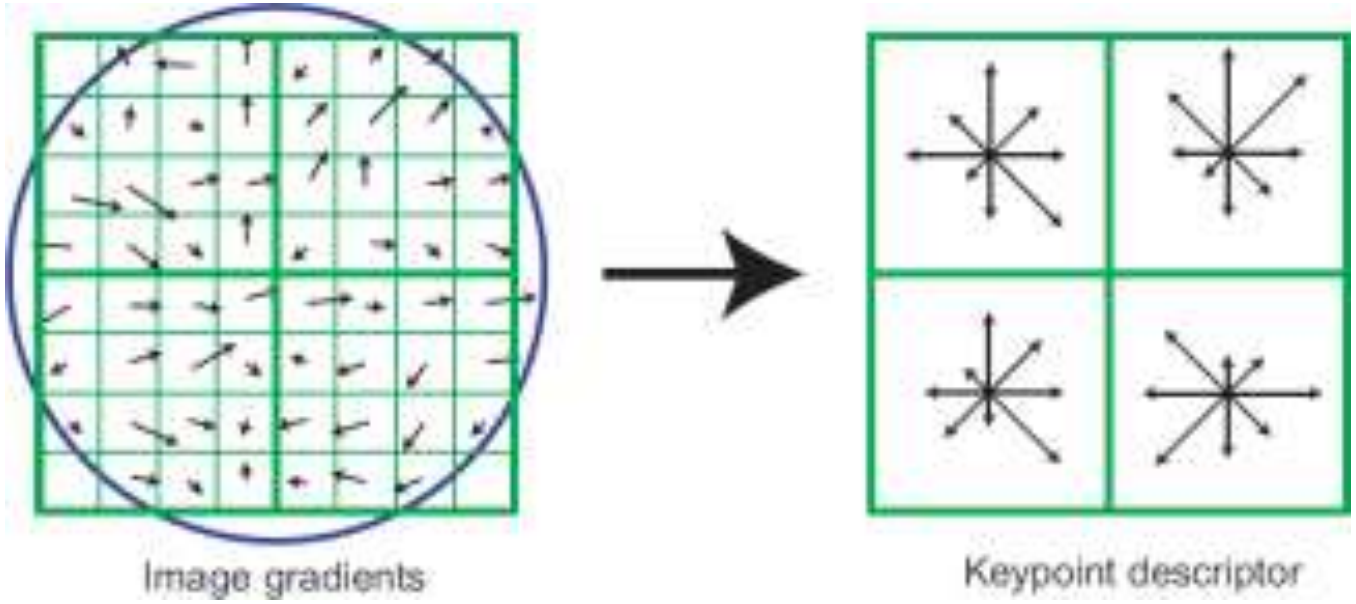


Image taken from D. Lowe, “Distinctive Image Features from Scale-Invariant Points”, IJCV 2004

Invariances of SIFT descriptor

- Scale, translation and rotationally invariant

- Regarding illumination
  - The SIFT descriptor (so far) is not illumination invariant – the histogram entries are weighted by gradient magnitude.
  - Hence the descriptor vector is normalized to unit magnitude. This will normalize scalar multiplicative intensity changes.
  - Scalar additive changes don't matter – gradients are invariant to constant offsets anyway.
  - Not insensitive to non-linear illumination